Langage Interprété

Modularisation et organisation de projet

1ère année Cycle ingénieur ISEN – Lille

2026/2027

Abderrahmane Kasmi

Séquence 3

Modularisation et organisation de projet

01 **Imports**

02 **Modules**

03 **Packages**

04 **Architecture d’un projet**

01

Imports

Syntaxes d'importation

- **Import global**

```
import math

print(math.sqrt(16))
```

- **Import ciblé**

```
from math import sqrt

print(sqrt(16))
```

Les alias

Ce sont des raccourcis ou des surnoms donnés aux bibliothèques. Ils servent à :

- **Raccourcir le code:** éviter de répéter les noms des bibliothèques à chaque utilisation
- **Eviter les conflits des noms des bibliothèques**
- **Faciliter le remplacement d'un outil:** En utilisant le même alias pour le nouvel outil, on évite de modifier le reste du code
- **Respecter les conventions de la communauté**

Les alias

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

dataframe = pd.read_csv("data.csv")
```

`sys.path`

Une liste de chemins dans lesquels Python va chercher les bibliothèques importées, elle se constitue de:

- **Le dossier des bibliothèques standards**
- **Le dossier des bibliothèques tierces**
- **Le dossier courant:** répertoire qui contient le script exécuté

Python parcourt la liste des chemins (`sys.path`) **dans l'ordre**, cherche le module importé et **s'arrête dès qu'il le trouve**

Comment Python charge un module ?

- Pour chaque bibliothèque importée, python exécute le fichier .py en entier même si l'import cible une seule fonction
- Le module chargé est stocké dans le dictionnaire `sys.modules` (pour éviter de le recharger une seconde fois).
- Python crée une variable locale (le nom du module) dans votre script pour vous donner accès à son contenu.
- En cas d'alias, l'alias est utilisé comme nom de variable

02

Modules

Un module, c’est quoi ?

C’est un fichier Python (.py) qui contient du code — des fonctions, des classes ou des variables — **réutilisable** dans d'autres programmes

Un fichier Python (.py) peut être exécuté en tant que fichier ou en tant que module:

- **Comme fichier**

```
$ python hello.py
```

- **Comme module**

```
$ python -m hello
```

La variable `__name__`

C’est une variable spéciale intégrée (dunder variable) qui contient **le nom du module en cours d'exécution**. Sa valeur dépend de la façon dont le fichier est exécuté :

- **Le fichier est exécuté directement**

  `__name__` sera égale à `"__main__"`

- **Le fichier est importé**

  `__name__` sera égale au nom du module (le fichier .py sans extension)

La variable __name__

Un bloc de code très courant en python:

```
if __name__ == "__main__":

        print("Script lancé directement")
```

Il permet d'insérer un code qui s'exécute seulement si le fichier est lancé directement

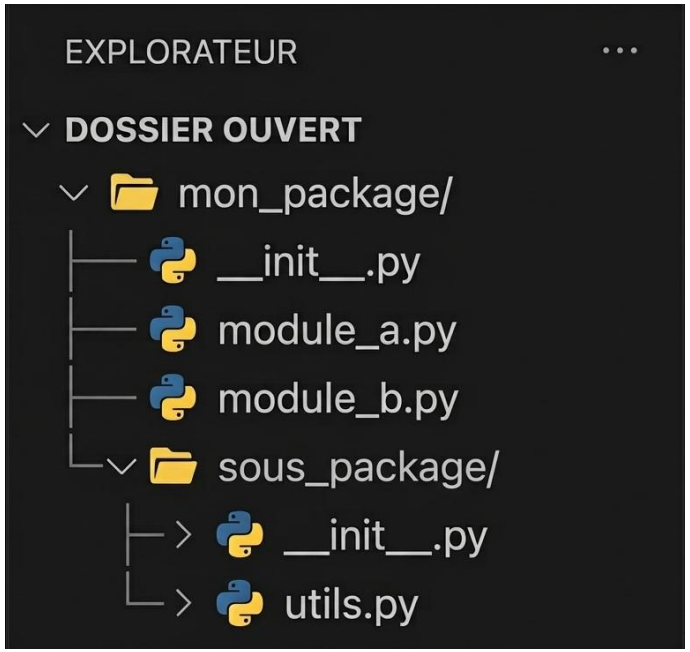
03

Packages

Un package, c’est quoi ?

C’est un dossier **regroupant plusieurs modules** Python, généralement **identifié par un fichier __init__.py**, permettant d'organiser le code de manière hiérarchique.

- Un package peut contenir des sous-packages



`__init__.py`

C’est un fichier qui sert de port d’entrée du package

- Le fichier s’exécute lors de l’importation des packages
- Il permet de simplifier les imports en exposant directement les fonctions, classes ou variables principales à **la racine du package**
- Pour les anciennes versions de python, l’existence du fichier était indispensable pour désigner le dossier comme package python, maintenant le fichier est souvent vide.

Chemins absolus et chemins relatifs

Pour importer un module dans un autre, le développeur peut spécifier **le chemin absolu** (à partir du paquet racine), ou bien **le chemin relatif au module importateur**.

- **En chemin absolu:** Python cherche le module dans la liste des chemin définis par `sys.path`
- **En chemin relatif:** Python doit connaître l'arborescence du package pour pouvoir trouver les modules importés

La variable `__package__`

C’est une variable spéciale qui indique à python à quel paquet appartient le module courant

- En cas d’importation avec chemins relatifs, python se base sur cette variable pour connaître l’hiérarchie

- `__package__` d’un fichier python exécuter directement (`python test.py`) contient `None`

Une bibliothèque, c’est quoi ?

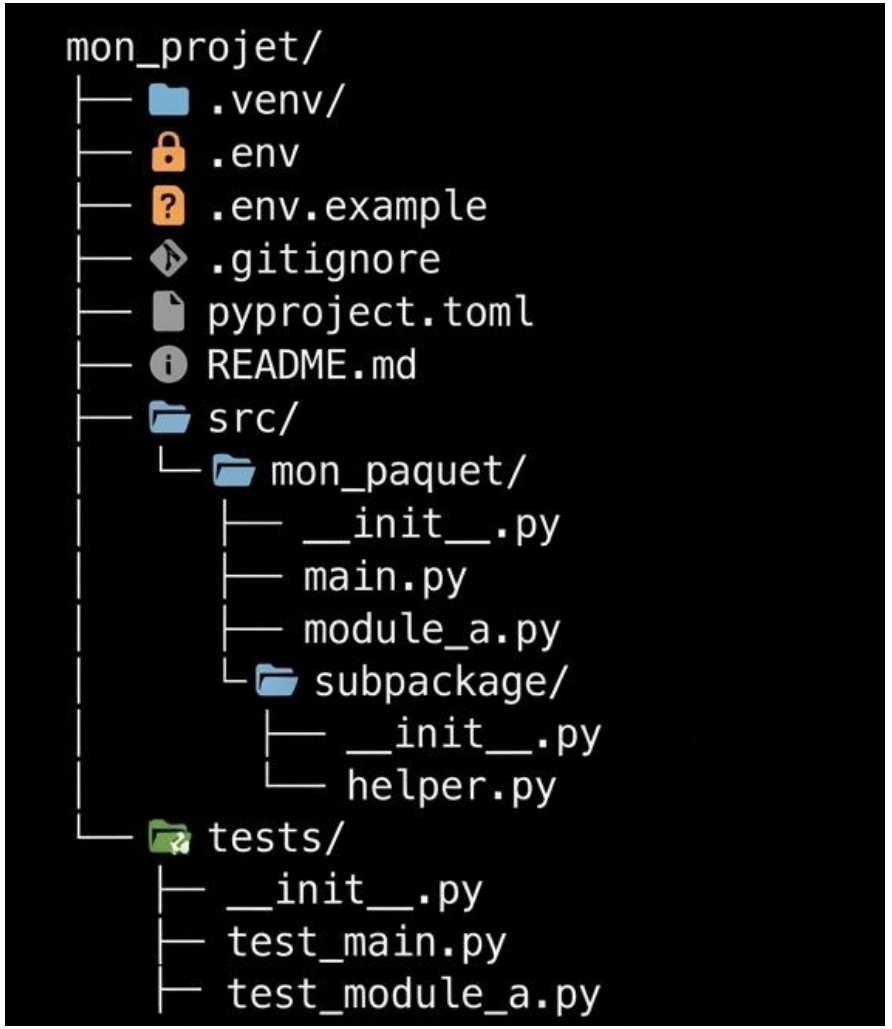
C’est juste un terme générique qui désigner tout ensemble de **code réutilisable** (modules ou paquets) mis à disposition des développeurs

04

Architecture d’un projet

Architecture d’un projet python

Il n’y a pas de règles imposées par Python pour structurer un projet. Il existe cependant des **conventions** largement utilisées pour rendre les projets plus faciles à **installer, tester, maintenir et distribuer.**



Architecture d’un projet python

- **.venv:** C'est un dossier qui contient tout ce qui concerne l'environnement virtuel du projet (séquence 4)
- **.env:** C'est un fichier où l'on stocke les variables d'environnement et les secrets (ex. clés d'API)
- **.env.example:** C'est un fichier modèle qui montre la structure de .env (sans les données sensibles) pour les développeurs souhaitant installer le projet
- **.gitignore:** C'est un fichier où l'on spécifie les fichiers et dossiers à ignorer par Git qui ne doivent pas être partagé (.env par exemple)
- **pyproject.toml:** C'est le fichier de configuration central du projet Python. Il définit la gestion des dépendances, les outils de build et la configuration des outils

Architecture d’un projet python

- **README.md:** C'est le fichier principal de documentation expliquant la présentation du projet, son installation et son utilisation.
- **src:** Le dossier principal contenant le code source du projet (paquets et modules)
- **tests :** Le dossier où sont regroupés les tests automatisés (sa structure reflète généralement celle du dossier `src`).